

# Cierre de Sprint – Evidencia Metodológica RUP + SCRUM

---

## Información general del Sprint

- **Sprint:** Sprint No. 1
  - **Periodo:** 14/Ene/2026 – 30/Ene/2026
  - **Objetivo del Sprint:**  
El objetivo funcional es el de programar las mecánicas bases, de tal forma que se tenga el cimiento del gameplay. Esta etapa produce componentes, clases y prefabs (Entre otros objetos) aprovechables y reusables para implementar los distintos puzzles e interacciones que se llevarán a cabo en el juego, posteriormente. Funcionalmente, produjo: El movimiento del jugador y el siguiente flujo de puzzles: Ver un halo en el área interactuable del puzzle, dar tap sobre el puzzle, ver (0 no) el tutorial del puzzle, jugar el puzzle, ganar o perder el puzzle y, finalmente, desactivar el puzzle.
- 

## Matriz de Trazabilidad

Fase RUP ↔ Objetivos ↔ Actividades ↔ Artefactos

 Tabla de trazabilidad

| Fase RUP | Objetivo de la Fase | Actividades realizadas en el Sprint | Artefactos producidos |
|----------|---------------------|-------------------------------------|-----------------------|
|          |                     |                                     |                       |

|              |                                                                               |                                                                                                                                                                                                               |                                                                                                                               |
|--------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Incepción    | Definir el alcance del sprint.                                                | Definición y desglose de las Épicas 1 y 2. Establecimiento del flujo lógico del sistema de puzzles y movimiento del jugador.                                                                                  | Product Backlog actualizado, Diagramas de clases, de secuencia y de casos de uso (actualizados).                              |
| Elaboración  | Actualizar la arquitectura base, aplicando principios SOLID.                  | Diseño de interfaces de comportamiento (ITappable, IPuzzlePausable). Definición del Patrón Observer en el GameManager. Diseño de límites de cámara y jugador.                                                 | Diagrama de Clases UML (actualizado), Modelo de Arquitectura de Software.                                                     |
| Construcción | Desarrollar de forma iterativa funcionalidades y sistemas clave del proyecto. | Programación del script de movimiento 2D. Implementación de la cámara con damping. Creación del sistema de flujo de puzzles (Área de trigger -> tap -> tutorial (Condicional) -> gameplay -> ganar o perder). | Scripts de Unity (.cs), Prefabs de Jugador/Cámara, PuzzleManager, PuzzleHalo, Background, etc. Escenas de gameplay funcional. |

|            |                                                               |                                                                                            |                                          |
|------------|---------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------|
| Transición | Asegurar que el software cumpla con los requisitos esperados. | Pruebas para con el motor de Unity para ver si el comportamiento coincidía con el deseado. | Ramas con funcionalidades implementadas. |
|------------|---------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------|

---

### 3 Evidencia de decisiones de diseño por fase

#### Justificación técnica

En este Sprint se tomaron las siguientes decisiones relevantes:

#### Decisiones arquitectónicas

- **Decisión:** Orquestación de estados de pausa mediante el patrón Observer y la interfaz IPuzzlePausable.
- **Fase RUP asociada:** Elaboración / Construcción (Implementación Visual).
- **Justificación técnica:** Mitigación de riesgos de efectos secundarios. Al entrar en un puzzle, el sistema debe garantizar que el PlayerMovement y otros elementos no interfieran. El patrón Observer permite que el GameManager notifique el cambio de estado sin tener referencias directas a clases concretas, lo que permite que se extienda su funcionalidad a notificar otras clases, cumpliendo con el Open/Closed Principle (OCP).
- **Impacto en el sistema:** Mantenibilidad y Robustez. Facilita la adición de nuevos elementos que deban reaccionar a la pausa del puzzle sin generar código espagueti en el GameManager.

## Decisiones arquitectónicas

- **Decisión:** Implementación de un sistema de detección mediante `InputHandler` -> `RayCaster` -> `ITappable`.
- **Fase RUP asociada:** Elaboración / Construcción (Implementación Visual).
- **Justificación técnica:** Se aplica el principio de Inversión de Dependencias (DIP). El `InputHandler` no necesita conocer la lógica interna del objeto tocado; solo necesita saber que cumple con el contrato de `ITappable`. El uso de Raycasting permite una detección espacial precisa en entornos 2D/3D sin acoplar la UI al espacio de juego.
- **Impacto en el sistema:** Permite añadir infinitos objetos interactivables (puzzles, easter eggs, notas) sin modificar el código de detección de entradas.

## Decisiones arquitectónicas

- **Decisión:** Creación de un `ControlManager` para el mapeo centralizado de entradas y acciones.
- **Fase RUP asociada:** Elaboración / Construcción (Implementación Visual).
- **Justificación técnica:** Centralización de la configuración de controles. Esto separa la "intención" del jugador (ej. "Interactuar") de la "tecla/gesto" física. Es un paso previo esencial para la portabilidad del software.
- **Impacto en el sistema:** Permite cambiar o remapear controles en el futuro para mejorar la accesibilidad o portar el juego a otras

plataformas sin afectar la lógica de los sistemas que reciben los comandos.

## Decisiones arquitectónicas

- **Decisión:** Implementación de PuzzleHalo y un Background común para el encuadre del área jugable.
- **Fase RUP asociada:** Elaboración (Diseño de Interacción) / Construcción (Implementación Visual).
- **Justificación técnica:** Uso de Affordance y feedback visual preventivo. El Halo indica al usuario que el objeto es "accionable" antes de que realice la acción, reduciendo la carga cognitiva y el error del usuario. El background común establece un modelo mental claro sobre los límites del puzzle.
- **Impacto en el sistema:** Mejora la Usabilidad y Experiencia de Usuario (UX), mediante potenciar la claridad del Serious Game, asegurando que el estudiante se concentre en el contenido de bioseguridad y no en entender cómo interactuar con la interfaz.

## Organización del sistema

- **Separación aplicada:**
  - Core
  - Sistema de Puzzles
  - Persistencia
- **Motivo:** Aplicar buenos principios de programación, de tal forma que se cree un código estructurado y flexible para modificación que evita código spaghetti

- **Beneficio esperado:** Crear y diseñar clases más reutilizables, flexibles y mantenibles o sostenibles. Esto se logra mediante el uso de los principios SOLID que promueven el acoplamiento débil o flojo, lo que impacta en las clases, haciendo que sean menos dependientes entre sí, minimizando los cambios que se requieren en una clase, cuando se necesita modificar otra. Esto es, que al requerir extender las funcionalidades de una clase, se evita modificar y reparar otras clases de sobre manera, permitiendo que los cambios y modificaciones se queden en una clase o la menor cantidad de clases posibles.

## Patrones o principios utilizados

- **Patrón / Principio:** Observer / Dependency Inversion / Open-Closed Principle.
  - **Contexto de uso:**
    - Observer -> En la notificación hacia los objetos que se deben pausar al entrar (Interactuar) en un puzzle.
    - Dependency inversion -> En el diseño de la interacción entre clases, procurando que las clases dependan o se fien de interfaces en lugar de clases concretas.
    - Open-Closed Principle -> En la implementación del patrón Observer, que permite extender la funcionalidad de la clase GameManager, para notificar otras clases, sin tener que modificar su comportamiento base.
  - **Fase RUP:** Elaboración / Construcción
-

## Bitácora resumida por fase RUP

### Incepción

#### Qué se hizo:

Se definió el alcance del sprint y se desglosaron las Épicas 1 y 2, estableciendo el flujo base del sistema de movimiento y del sistema de puzzles (detección, interacción, tutorial, gameplay y cierre).

#### Por qué se hizo:

Se necesitaba clarificar objetivos, dependencias y límites funcionales del sprint para evitar desviaciones durante la construcción y garantizar que el desarrollo se enfocará en el núcleo del gameplay.

#### Resultado:

Objetivos y alcance del sprint definidos, backlog actualizado y flujo lógico base del sistema validado para iniciar implementación.

### Elaboración

#### Qué se hizo:

Se diseñó la arquitectura base del sistema aplicando principios SOLID, definiendo contratos de comportamiento mediante interfaces (ITappable, IPuzzlePausable) y estructurando la comunicación entre sistemas con el patrón Observer a través del GameManager.



### **Por qué se hizo:**

Se detectó la necesidad de reducir el acoplamiento entre clases y prevenir efectos secundarios (por ejemplo, movimiento activo durante puzzles), asegurando escalabilidad para futuros puzzles, interacciones y sistemas.



### **Resultado:**

Base arquitectónica validada con componentes reutilizables, extensibles y con comunicación desacoplada, lista para implementarse en Unity.

## **Construcción**



### **Qué se hizo:**

Se programaron las mecánicas fundamentales: movimiento 2D del jugador, integración de cámara con damping, y el flujo completo del sistema de puzzles (trigger → interacción → tutorial condicional → gameplay → victoria/derrota → desactivación). Además, se crearon prefabs y escenas funcionales.



### **Por qué se hizo:**

Se requería un cimiento técnico y funcional sólido que permitiera implementar posteriormente los distintos puzzles e interacciones del juego sin rehacer sistemas base.

## **Resultado:**

Prototipo funcional con movimiento, cámara y sistema de puzzles operativo, con scripts y prefabs reutilizables listos para expansión.

## **Transición**

### **Qué se hizo:**

Se realizaron pruebas dentro del motor Unity para validar que el comportamiento del sistema coincidiera con el diseño esperado, verificando el flujo de interacción y la correcta pausa/reanudación de componentes.

### **Por qué se hizo:**

Era necesario confirmar que las implementaciones respondieron correctamente en tiempo real, reduciendo riesgos técnicos y evitando fallos de integración para los siguientes sprints.

## **Resultado:**

Comportamiento validado mediante pruebas en Unity y ramas con funcionalidades implementadas listas para continuar el desarrollo iterativo.

## **5** Validación del incremento (si aplica)

- **Funcionalidad:** <OK / Observaciones>
- **Fiabilidad:** <OK / Observaciones>
- **Rendimiento:** <OK / Observaciones>
- **Usabilidad:** <OK / Observaciones>